ARGUS: A Context-Aware Software Architecture for Smart Environments

Anonymous Authors

***Abstract*—The paradigm of Smart Environments (SE) has transitioned from simple remote automation to proactive, context-aware ecosystems driven by Artificial Intelligence. However, orchestrating heterogeneous AI subsystems, which range from high-frequency sensor streams to computationally intensive Large Language Models (LLMs), imposes significant challenges regarding latency, interoperability, and data privacy. To address these issues, we present ARGUS, a distributed, event-driven software architecture designed to orchestrate intelligent decision-making across the Edge-Cloud continuum. We validate the architecture through a reference implementation and performance evaluation, demonstrating that ARGUS effectively accommodates heavy computational workloads, such as those used in generative AI and recommendation systems, to create intelligent environments that dynamically adapt to user needs.**

***Index Terms*—Smart Environments, Edge-Cloud Continuum, Microservices, Large Language Models, Event-Driven Architecture, Internet of Things**

I. Introduction

In the last few decades, Smart Environment (SE) technology has advanced substantially in both industry and research [1]. The concept of physical environments that interact intelligently and unobtrusively with people has gained significant attention, largely due to the rapid expansion of Internet of Things (IoT) technologies and the increasing affordability of connected devices [2], [3]. In such smart environments, users can monitor and control IoT devices individually or through automation rules via smartphones and other interfaces, achieving finer-grained control over their surroundings while enhancing security, comfort, and accessibility. These innovations are particularly promising for diverse user groups, including people with disabilities and older adults, for whom adaptive environments can support greater autonomy and safety [4], [5].

A central challenge in this landscape is designing context-aware solutions capable of running real-time recommendation and decision-making systems within smart environments [6]. In fact, beyond simple *If-This-Then-That* rules, next-generation smart environments must integrate heterogeneous subsystems so that they can learn from user behavior, continuously modify their internal configuration, and proactively adapt the environment to predicted user needs [1], [5]–[7].

This study leverages a reference implementation developed in collaboration between a large industry partner and our research group. We deployed an AI-oriented smart home that utilizes machine learning to suggest device actions based on inferred living habits, thereby creating a cohesive, intelligent behavioral layer [1]. Illustrative scenarios include context-aware lighting, where opening a door prompts a recommendation to activate external lights, while closing it suggests deactivation, and synchronized maintenance, where opening a window signals the system to deploy a robot vacuum, automatically concluding the cleaning cycle when the window is closed.

Realizing these scenarios requires an architecture capable of reconciling modern *Ambient Intelligence* (AmI) conflicts: specifically, the tension between massive data processing demands—such as analyzing high-bandwidth video streams or running generative AI models—and the strict latency constraints of real-time device control. Furthermore, the system must ensure privacy by design, processing sensitive visual data at the edge rather than offloading it to centralized clouds, while simultaneously providing robust orchestration to ensure that "slow" reasoning agents do not block "fast" reactive loops. Consequently, this research is driven by two primary questions:

RQ1 *Is it possible to orchestrate heterogeneous AI modules for Ambient Intelligence by effectively combining Edge and Cloud functionality?*
This question addresses the architectural tension between the massive computational resources required by generative models—such as LLMs and Computer Vision—and the strict low-latency constraints necessary for Ambient Intelligence and it's usability requirements.

RQ2 *How do throughput limits interact with processing latency when implementing synchronous incremental learning in an event-driven architecture?*
This inquiry is driven by the performance costs of the system's "proactive adaptation" capability. Since incremental learning algorithms must update internal models in real-time, they introduce blocking operations into the event loop; thus, it is critical to quantify the precise point at which model training saturation begins to degrade system responsiveness.

To answer these questions, we propose ARGUS, a distributed architecture that integrates real-time decision-making modules—such as machine learning systems and large language models—with smart IoT devices operating at the edge. ARGUS enables context-aware, closed-loop control in smart environments by orchestrating services across the Edge-Cloud continuum. The main contributions of this paper are summarized as follows:

Identify applicable funding agency here. If none, delete this.

[1]Due to Non-Disclosure Agreement (NDA) restrictions with our industrial partner, the full source code and specific artifacts exposing proprietary implementation details could not be made publicly available.

- We define an event-driven microservices architecture that successfully integrates distinct AI paradigms—from deterministic computer vision to generative Large Language Models (LLMs)—without compromising the latency of safety-critical control loops.
- We demonstrate a mechanism for processing high-bandwidth video streams locally at the edge, identifying critical threats (*e.g.*, weapons or intruders) in real-time while ensuring sensitive data never leaves the local network.
- We provide a quantitative analysis of the throughput and latency trade-offs involved in deploying synchronous incremental learning systems for proactive home automation, identifying specific saturation points in sequential processing pipelines.

II. Background and Related Work

The transition from basic home automation to fully autonomous Smart Environments (SE) requires a convergence of distributed systems, robust middleware, and advanced artificial intelligence. In this section, we first provide background on the architectural foundations of the Edge-Cloud continuum, the evolution of intelligent agents in residential settings, and the communication patterns required for real-time orchestration (Sections II-A–II-C). We then discuss how representative systems in these areas relate to our proposal and highlight the remaining gaps that motivate the ARGUS architecture (Section II-D).

A. Architectural Paradigms: The Edge-Cloud Continuum

A smart home is defined not merely by connectivity, but by its ability to enhance resident autonomy through intelligent interaction [6]. While early implementations relied on monolithic local controllers [8], modern architectures are increasingly distributed across the Edge-Cloud continuum [9].

Industrial surveys indicate that while cloud deployments predominate, accounting for approximately 75% of computational capacity in typical IoT architectures, there is a critical reliance on the edge (approximately 25%) for tasks requiring low latency and privacy preservation [9]. This hybrid approach is driven by four competing quality attributes: reliability, performance, security, and cost [9].

To manage this distributed complexity, modern systems increasingly adopt the Microservices architectural style [10]. By decomposing monolithic controllers into independently deployable services, architects can isolate failures and scale components independently. In the context of SE, this allows resource-intensive modules (such as computer vision) to run on specialized edge hardware, while management interfaces reside in the cloud, integrated via API Gateways [11].

In existing smart-home platforms, these microservice and API Gateway patterns are typically employed to modularize device management and user-facing interfaces, while treating AI components as opaque services. However, they rarely make the orchestration of heterogeneous AI workloads—such as computer vision, recommendation systems, and LLM-based agents—a first-class architectural concern across the Edge-Cloud continuum, particularly under strict latency and privacy constraints. In contrast, ARGUS builds on this microservices foundation but explicitly models the life-cycle and placement of these diverse AI subsystems, as detailed in Section IV.

B. From Reactive Rules to Agentic Reasoning

The "intelligence" in Ambient Intelligence (AmI) has evolved significantly. First-generation systems relied on simple Rule-Based Systems (RBS) or trigger-action paradigms (e.g., IFTTT), which often proved brittle in dynamic environments [6], [12].

Second-generation systems introduced classical Machine Learning (ML) and Deep Learning (DL) to model user behavior patterns [13]. Research has demonstrated the efficacy of models such as Long Short-Term Memory (LSTM) networks for capturing temporal sequences in sensor data, enabling proactive recommendations rather than purely reactive controls [6], [12]. However, these models often lack flexibility when facing novel situations or ambiguous user intents.

Recent advancements in Large Language Models (LLMs) have enabled a third generation: *Agentic Systems* [14]. Unlike static classifiers, LLM-based agents can employ natural language planning to decompose complex, abstract goals into executable steps [15], [16]. Emerging patterns such as ReAct (Reasoning and Acting) allow agents to interleave reasoning traces with calls to external tools, verifying intermediate conditions before proceeding [17], [18]. By integrating episodic and semantic memory structures, these agents can maintain long-term context, transforming the smart home from a probabilistic predictor into a reasoning partner [19], [20].

Most of these works focus on the design of the agent and its memory mechanisms in isolation, assuming an abstract tool interface and leaving the underlying smart-home infrastructure largely unspecified. ARGUS complements this line of research by embedding LLM-based agents into a concrete, event-driven microservices architecture, where reasoning traces trigger and coordinate multiple perception and actuation services without blocking low-latency control loops.

C. Data Exchange and Event-Driven Middleware

The efficacy of distributed AI depends heavily on the underlying data exchange mechanisms. The Publish/Subscribe (pub/sub) paradigm, mediated by protocols such as Message Queuing Telemetry Transport (MQTT), has become the de facto standard for decoupling sensors from reasoning engines [6], [9], [21].

However, traditional brokers often lack the flexibility required for heterogeneous AI workloads. High-frequency sensor data (e.g., accelerometers) and high-bandwidth streams (e.g., video metadata) compete for bandwidth [21]. Research into *Adaptive Message Broker Architectures* suggests that regulating data flows at the subscription level, using AI planning to dynamically adjust priorities and drop rates, can significantly improve Quality of Service (QoS) [21].

Furthermore, data heterogeneity poses a challenge for downstream learning. A single device message often encodes multiple state properties (e.g., brightness, color, and temperature). Effective architectures must therefore implement robust normalization layers that transform raw, vendor-specific payloads into structured tabular representations suitable for multi-label classification and incremental training [22].

D. Related Work and Gap Analysis

To the best of our knowledge, existing systems typically address architecture, AI reasoning, and middleware in isolation: architectural work focuses on modularity and scalability but treats AI workloads as black-box services; AI-oriented work emphasizes models and agents but assumes an abstract infrastructure; and middleware work optimizes data flows without integrating high-level reasoning and recommendation modules.

ARGUS is proposed to bridge these gaps by providing an end-to-end software architecture that (1) orchestrates multiple AI subsystems—including computer vision, recommendation, and LLM-based agents—across the Edge-Cloud continuum, (2) enforces privacy-by-design for high-bandwidth sensing by colocating sensitive processing at the edge, and (3) exposes a unified, event-driven interface that allows both traditional ML models and agentic systems to consume the same normalized representation of the environment. The next section details the design principles and concrete services that realize this architecture.

III. Research Design

This study follows a multi-stage research design to investigate how heterogeneous AI modules can be orchestrated for Ambient Intelligence across the Edge–Cloud continuum. to answer our research questions, we planned a total of four stages.

Literature Review. We first conducted a targeted literature review on (i) software architectures for IoT and the Edge–Cloud continuum, (ii) AI-based automation and agentic systems for smart homes, and (iii) pub/sub middleware for heterogeneous sensor data, as summarized in Section II. This synthesis revealed that existing work typically treats architecture, AI reasoning, and middleware in isolation, motivating the need for an end-to-end architecture that explicitly supports heterogeneous AI subsystems and incremental learning, thus shaping RQ1 and RQ2.

Discussions with Experts and Industrial Partners. We then held structured discussions with domain experts and practitioners from our industrial partner to elicit real-world constraints regarding device ecosystems, privacy requirements, deployment topologies, and user experience. The resulting information was key for the definition of the functional and non-functional requirements, later consolidated in Table I, refined the architectural goals associated with RQ1.

Exploratory Prototyping. Next, we developed small-scale prototypes of key components, including message-broker configurations, an initial recommendation pipeline with incremental learning, and a basic computer vision service. These prototypes exposed practical trade-offs between end-to-end latency, throughput, and resource utilization in an event-driven setting, informing design choices such as asynchronous communication and edge placement of high-bandwidth processing, which are central to RQ2.

Architecture Definition and Evaluation. Finally, we consolidated the insights from the previous stages into the ARGUS architecture, detailed in Sections IV and V, and designed an empirical evaluation focused on vision inference and recommendation performance (Section VI). In this way, RQ1 is addressed by demonstrating a concrete architecture and reference implementation that orchestrate heterogeneous AI modules across edge and cloud, while RQ2 is addressed by quantifying the interaction between throughput and latency in the synchronous incremental learning pipeline embedded in our event-driven architecture.

IV. ARGUS: Architecture and Design Principles

To address the challenges of integrating heterogeneous AI systems with real-time IoT constraints, we propose ARGUS: a modular, distributed architecture designed for the edge-cloud continuum. ARGUS is not a monolithic application but a set of microservices orchestrated to provide context-aware decision-making.

A. Architectural Requirements and Design Drivers

The architectural design of ARGUS is governed by a set of requirements derived from the limitations of existing smart environment frameworks. These requirements, detailed in Table I, ensure that the system provides advanced intelligence while maintaining the reliability and usability necessary for domestic adoption. They serve as the primary design drivers, influencing the selection of the microservices pattern and the hybrid deployment topology.

B. Architectural Style and Design Principles

To satisfy these requirements, ARGUS adopts a *Microservices Architecture* with an *Asynchronous Event-Driven* communication pattern [23]. This decoupling is critical for latency isolation, ensuring that heavy inference tasks, such as an LLM reasoning trace, do not block high-frequency sensor processing. Furthermore, this approach ensures technology agnosticism, allowing the use of diverse technology stacks tailored to each domain, such as Python for Machine Learning and Rust for high-performance middleware.

C. Context Level (Level 1)

At the highest level of abstraction, ARGUS acts as an intelligent intermediary between the physical environment and the resident. The system operates within a context defined by three primary entities. First, **Residents and Administrators** interact with the system via natural language (voice/text) or administrative dashboards to query state or issue overrides. Second, the system integrates with **Vendor IoT Clouds** to manage standard commercial devices like switches and thermostats; since many commodity devices lack local APIs,

TABLE I
FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS DRIVING THE ARGUS ARCHITECTURE.

ID	Requirement Type	Description	Design Driver (User Advantage)
Functional Requirements			
FR1	Device Integration	Normalization of diverse protocols (Cloud APIs, RTSP, MQTT) into a unified event schema.	Eliminates vendor lock-in; allows mix-and-match of best-in-class devices.
FR2	Multimodal Perception	Detection of entities and events from high-bandwidth video streams and scalar sensors.	Enables the environment to "see" context beyond simple binary triggers.
FR3	Proactive Recommendation	Prediction of future actuator states based on historical behavior and temporal context.	Automates routine tasks without requiring manual rule configuration.
FR4	Agentic Reasoning	Support for natural language planning and tool execution via Large Language Models.	Handles complex, unscripted scenarios (e.g., "secure the house") dynamically.
FR5	Dynamic Orchestration	Dynamic lifecycle management and workload distribution for inference nodes.	Ensures optimal resource usage across distributed hardware.
FR6	Observability & Admin	Real-time configuration and visual monitoring of distributed system health.	Prevents black-box syndrome; increases trust and manageability.
Non-Functional Requirements			
NFR1	Low Latency	Handle requests and device updates in low latency.	Guarantees "smart" features do not degrade fundamental home utility.
NFR2	Privacy by Design	Local processing of sensitive high-bandwidth data (specifically raw video) at the edge.	Protects the data from residents.
NFR3	Scalability	Support for horizontal addition of edge nodes and sensor inputs.	allows the system to handle many smart environments simultaneously.
NFR4	Resilience	Automatic recovery from transient failures in reasoning agents or cloud APIs.	Ensures continuous operation of core functions during outages.
NFR5	Interactive Usability	Support for intuitive, multimodal interaction (Voice/Chat) for non-technical users.	Increases accessibility for diverse user groups (e.g., elderly).
NFR6	Deployment Flexibility	Support for heterogeneous infrastructure topologies (Edge, On-Premise, Cloud, or Hybrid).	Allows deployment optimization for cost, privacy, or hardware availability specific to the site.

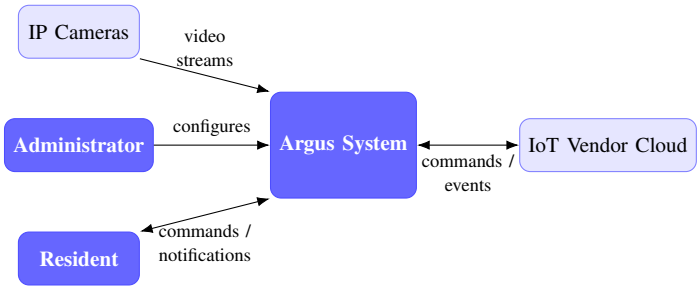

Fig. 1. Context Diagram of the ARGUS architecture, showing Residents, Administrators, Vendor Cloud, and Cameras interacting with ARGUS.

ARGUS exchanges command payloads with these external clouds. Finally, **High-Bandwidth Local Sensors**, such as IP Cameras, connect directly to ARGUS Edge nodes via RTSP or WebRTC. This direct link bypasses external clouds entirely, a critical design choice that ensures low latency and preserves the privacy of sensitive video data. Figure 1 shows the architecture in this level of abstraction.

D. Container Level (Level 2)

ARGUS is internally composed of containerized services organized into four logical layers, as illustrated in Figure 2.

1) Layer 1: Data Infrastructure and Persistence: The backbone of the system is the Data Infrastructure Layer, which handles state management and inter-service communication. The central communication service is the Message Broker (Eclipse Mosquitto) [24], which implements the MQTT protocol. All sensor events, telemetry, and control commands are published here, allowing subsystems to react to state changes without direct coupling.

Complementing the broker, we employ a heterogeneous persistence strategy to match the data storage requirements of each service with the appropriate technology. We utilize MongoDB [25] as a document store for semi-structured data such as user profiles and event history, while Redis [26] provides an in-memory key-value store for high-speed synchronization of device states, ensuring sub-millisecond access for real-time components. Binary artifacts, including Machine Learning model weights and camera snapshots, are managed by MinIO (Object Storage) [27], and structured metadata for orchestration logic are housed in MariaDB [28].

2) Layer 2: Core Services: These services abstract the complexity of real-world devices and external interfaces. The **IoT Cloud Adapter** acts as the translation layer, normalizing heterogeneous payloads (e.g., vendor-specific JSON) into a standardized internal event schema, ensuring downstream consumers remain agnostic to hardware protocols. The **Orchestrator** sits at the center of the system, managing the lifecycle of distributed nodes and dynamically routing computational tasks. External access is managed by an API Gateway (NGINX [29]), which handles TLS termination, rate limiting, and routing.

3) Layer 3: The Intelligence Layer: This layer contains stateless, horizontally scalable reasoning engines that trans-

form raw data into insight. The **Computer Vision Subsystem** processes video streams to detect occupancy and entities. The **Recommendation Engine (RecSys)** performs incremental learning on event streams to predict future user needs. Finally, **AI Agents** leverage Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) to execute complex reasoning and multi-step automation planning.

4) Layer 4: Presentation: The interface layer comprises the **Chatbot Application**, written in Flutter, for multimodal resident interaction, and the **Management UI**, made with Flask [30], for system administration and visualization of telemetry data.

E. Deployment: The Edge-Cloud Continuum

A distinguishing feature of the ARGUS architecture is its design, enabling multiple deployment strategies (NFR6). Indeed, when considering factors like privacy and heterogeneous processing nodes, ARGUS services can be deployed on Edge Nodes, in the Cloud, and on On-Premise servers. For environments like Smart Homes, where very low latency is not required, computational power can be offloaded to the Cloud, enabling heavy tasks such as LLM inference and continuous video stream processing.

However, some IoT vendors require the use of their public cloud to control their devices. In this case, information is proxied through the vendor's Cloud, leveraging existing infrastructure for command propagation.

V. ARGUS SUBSYSTEMS

The ARGUS architecture is divided into six important subsystems which will be individually presented in this section.

A. Real-time Recommendation Engine

The **Recommendation Engine (RecSys)** establishes the system's proactive control loop by continuously analyzing event streams to predict future user needs. As illustrated in Figure 3, the module operates downstream from the IoT infrastructure. Raw sensor events are first normalized by the **Third-Party IoT API Adapter** into a unified internal representation and published to the message broker. This adapter layer decouples the recommender from specific vendor protocols, ensuring a reliable, ordered stream of standardized device events for consumption.

Upon consumption, the engine transforms the continuous event stream into a structured format suitable for supervised learning. Incoming events are converted into tabular records, where devices are categorized as either predictors (sensors) or control targets (actuators). These records are enriched via *temporal transformation*, which encodes timestamps into features (*e.g.*, time-of-day) and lagged states to capture sequential usage patterns. To handle the irregular frequency of IoT data, the system employs *dynamic window construction*. Instead of fixed time intervals, training windows are defined by event density, ensuring that each update batch maintains temporal coherence and sufficient information density.

These dynamic windows feed the *incremental model training* phase. To address the non-stationary nature of human habits, the model updates its weights online using the most recent window rather than retraining from scratch, allowing the system to adapt to behavioral drifts in real-time. Finally, the prediction module infers optimal actuator states based on the current context. These recommendations are published back to the message broker, closing the control loop by triggering immediate physical adjustments in the smart environment.

B. Edge Computer Vision Subsystem

The Computer Vision (CV) subsystem is a major subsystem located in the Intelligence Layer (see Figure 2). It serves as the primary high-bandwidth perception engine within the ARGUS architecture. It is designed to reconcile the critical tension between multimodal perception (FR2) and privacy-by-design (NFR2). Unlike cloud-based vision APIs that require transmitting sensitive footage over the internet, this module enables image processing at the edge (e.g., on local NVIDIA Jetson nodes), ensuring that raw video streams are fully processed locally. Our CV subsystem design is twofold, focusing on Inference Orchestration and on Resilience and Isolation.

Concerning to *inference orchestration*, the subsystem architecturally acts as a wrapper around the NVIDIA Triton Inference Server [2], providing a unified abstraction for heterogeneous models. It implements the following pipeline:

1) *Ingestion:* The module connects directly to local IP cameras via RTSP, bypassing the central message broker to avoid network congestion. It employs a "latest-frame" sampling strategy, discarding stale buffers to minimize processing latency.
2) *Inference:* Frames are submitted to Triton, which orchestrates the concurrent execution of multiple loaded models (e.g., YOLOv11 for object detection, VGGFace for face recognition, etc.). The subsystem supports dynamic model swapping at runtime via MQTT control messages, allowing the Orchestrator to load heavier models only when specific contexts require them.
3) *Publication:* The raw outputs from the models are aggregated and transformed into lightweight, structured JSON events (e.g., {`event: "person_detected", confidence: 0.95`}). Only these metadata packets are published to the MQTT bus, resulting in a reduction of network bandwidth usage by orders of magnitude compared to video streaming.

Regarding *resilience and isolation*, aiming to ensure continuous operation in safety-critical environments, the module implements fault isolation at the model level. If a specific neural network fails to converge or crashes (e.g., due to an out-of-memory error), the wrapper isolates the failure, allowing other active models to continue processing the stream. Furthermore, the subsystem includes automated watchdog mechanisms that re-establish RTSP connections upon stream

[2]NVIDIA Triton Inference Server is a multi-framework model serving system that provides a unified interface for loading and executing machine learning models in production environments. More information available at https://docs.nvidia.com/deeplearning/triton-inference-server.

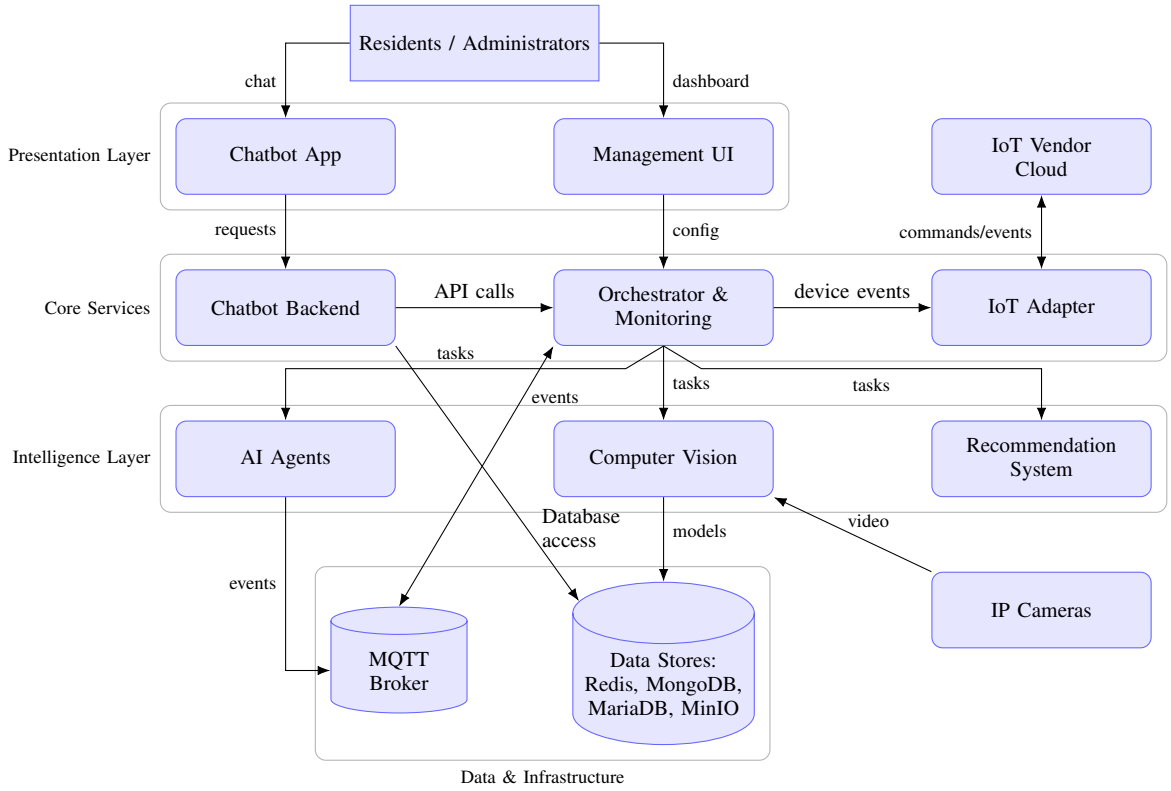

Fig. 2. The ARGUS simplified container-level architecture visualization. The diagram highlights the separation between the Data Infrastructure, Core Services, and the Intelligence Layer, connected via an MQTT Event Bus.

interruption, ensuring autonomous recovery without administrative intervention.

C. Agentic Reasoning and Planning

While the Recommendation Engine handles routine, pattern-based automation, the **AI Agents Subsystem** addresses complex, ambiguous, or unscripted scenarios (FR4). This module acts as the system's "Slow Thinking" engine, utilizing Large Language Models (LLMs) to interpret natural language intents, decompose abstract goals into actionable steps, and orchestrate the execution of other ARGUS subsystems.

To overcome the non-deterministic nature of LLMs, the architecture rejects monolithic "black box" agent implementations. Instead, reasoning workflows are modeled as *Directed Acyclic Graphs (DAGs)* executed via a low-code orchestration runtime (n8n). This graph-based design formalizes the *Chain-of-Thought* prompting technique [15], forcing the model to generate intermediate reasoning steps before committing to an action.

Furthermore, the system implements the *ReAct (Reasoning + Acting)* pattern [17]. Rather than hallucinating device states, the agents interleave reasoning traces with deterministic tool invocations. For example, to answer "Is the house secure?", an agent does not guess; it executes a *Perception Tool* (querying the Computer Vision subsystem), analyzes the returned structured metadata, and then formulates a grounded response. This "Tool Use" paradigm [18] effectively treats the rest of the ARGUS platform (Vision, IoT Adapter, Persistence) as a set of callable API functions exposed to the LLM.

To support long-horizon consistency and personalization, the agents employ a *Multi-Store Memory* architecture, inspired by recent research in generative agents [19], [20]. The state is partitioned into three distinct indices:

- *Episodic Memory:* A time-series log of past environmental events (ingested via MQTT) and prior agent actions. This allows the system to answer temporal queries (e.g., "Who was at the door yesterday?").
- *Semantic Memory:* A vector database containing embeddings of system documentation, policy rules, and entity descriptions. This utilizes *Retrieval-Augmented Generation (RAG)* [31] to ground agent responses in static factual knowledge.
- *Procedural Memory:* A collection of successful plan trajectories. When an agent successfully resolves a novel task, the sequence of actions is indexed, allowing the system to recall and adapt this "habit" for future similar requests without re-deriving the plan from scratch.

At last, given the high computational cost of LLM inference, the subsystem operates under strict resource quotas enforced by the Orchestrator. To mitigate latency, the architecture employs a caching layer for frequent queries and falls back to lighter, specialized models for simple tasks. Crucially, because the reasoning logic is encoded as explicit DAGs rather than opaque code, every step of the decision process—from the

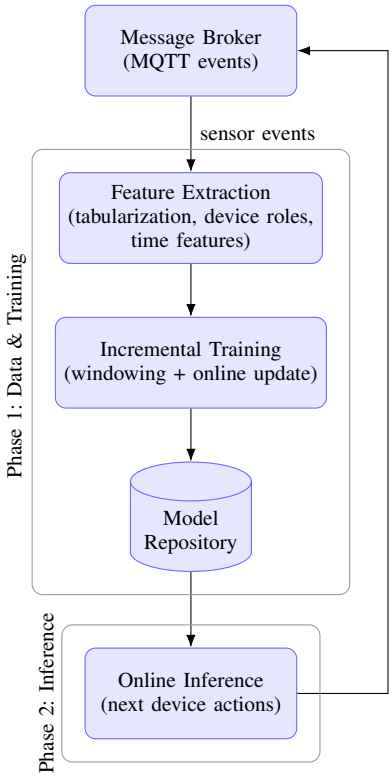

Fig. 3. Architecture of the smart-home recommender system. Events generated by sensors and actuators are sent to the IoT cloud, standardized by the Adapter, and forwarded via a message broker to the RecSys module. The RecSys module converts events into temporally structured tabular data, builds dynamic windows for incremental model training, and produces action recommendations that are returned to the broker for execution on actuators.

Fig. 4. Camera-captured input frame (top) and structured JSON output generated by the vision processing pipeline (bottom).

initial prompt to the final tool output—is recorded. This provides a complete audit trail (FR6), enabling administrators to inspect the "thought process" behind every automated decision and debug logic failures in the prompt chain.

D. Multimodal Chatbot Interface

The **Chatbot Interface** functions as the primary Human-Computer Interface (HCI) for the ARGUS solution, facilitating bidirectional communication between the resident and the smart environment. It serves a dual purpose: enabling the user to execute complex device commands via natural language (text or audio) and acting as the delivery channel for proactive insights generated by the Recommendation Engine. These recommendations can be solicited interactively during conversation or pushed asynchronously as notifications during safety-critical events.

1) Backend Architecture and NLP Pipeline: The subsystem is architected as a decoupled client-server model. The backend, implemented in *Python*, leverages a modular design centered on the *Flask* framework to expose RESTful endpoints for authentication and dialogue management. To support low-latency interactions, the system employs a polyglot persistence strategy: MongoDB [25] is utilized for long-term storage of conversational records and user profiles, while Redis [26] manages the real-time synchronization of device states across the ecosystem.

To enable robust natural language understanding, the backend integrates specialized Transformer-based architectures [32]. Furthermore, audio inputs are processed by an Automatic Speech Recognition (ASR) pipeline using the `Wav2Vec2-XLSR`[3] model. For semantic understanding, the system employs the `Serafim-100m`[4] sentence encoder to generate high-dimensional embeddings from user text. These embeddings are compared against a pre-computed vector space of device capabilities to resolve user intent.

2) Frontend and State Management: The client-side application is developed in *Dart* using the *Flutter* framework, ensuring cross-platform consistency. Furthermore, communication with the wider ARGUS architecture is hybrid: the frontend utilizes HTTP/REST for transactional operations with the backend (e.g., authentication) while establishing a continuous WebSocket subscription for device updates. This allows the interface to reflect the state of the physical environment in real-time without polling.

[3]Model: lgris/wav2vec2-large-xlsr-open-brazilian-portuguese
[4]Model: PORTULAN/serafim-100m-portuguese-pt-sentence-encoder-ir

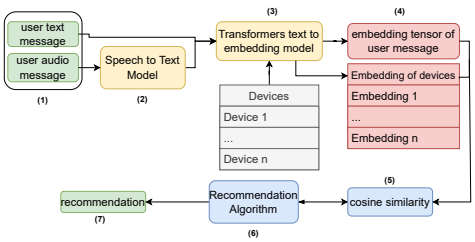

Fig. 5. Workflow of the Chatbot Recommendation Module. Audio input is transcribed via ASR, vectorized by a Transformer encoder, and compared with device embeddings via cosine similarity to generate actionable recommendations.

3) Operational Control Flows: Operationally, the interface manages two distinct control flows: *User-Initiated Command* and *Proactive Recommendation*.

In the *User-Initiated flow*, natural language inputs are processed as illustrated in Figure 5. The system resolves the user's intent against a dynamically updated registry of device attributes. Upon validation, the frontend forwards the action request to the **Third-Party IoT API Adapter**. This adapter translates the generic command into the specific payload structure required by the vendor's external cloud API, executing the change on the physical hardware.

In the *Proactive flow*, the logic is inverted. The Recommendation Engine detects a relevant context change (e.g., user arrival) and pushes a suggestion to the interface via the message bus. These appear as interactive notifications, allowing the user to approve or dismiss the system's proposed adaptation without needing to issue a prior command.

To support these flows, the synchronization module continuously monitors the event bus for device telemetry. It dynamically constructs a mapping of available devices and their mutable attributes (e.g., `light_color`, `brightness`), updating the Redis store in real-time. This ensures that the natural language models always reason over the valid, current state space of the environment, preventing hallucinations regarding non-existent devices or unsupported actions.

E. Management and Observability Subsystem

The **Management Interface** functions as the centralized control plane for the ARGUS ecosystem. While the Chatbot serves the end-user, this subsystem provides administrators with deep observability into system health, node lifecycle management, and data governance. It is architected to satisfy the requirement for dynamic configuration (FR6) by exposing a unified surface for visualizing real-time telemetry and executing CRUD operations on models, users, and metrics.

1) Architectural Design: The subsystem adopts a decoupled multi-service pattern via an API Gateway (NGINX [29]). The presentation layer is a server-side rendered application (built on Flask) that remains stateless to facilitate horizontal scaling, delegating session management to Redis [26]. It interacts with a specialized Storage Service (built on top of FastAPI [33]) which acts as the abstraction layer for the polyglot persistence stack. This design isolates the frontend from the complexities of the underlying database technologies (MongoDB [25] and MinIO [27]), enforcing a strict contract-first approach via OpenAPI specifications.

2) Observability Pipeline and Flow Reconstruction: A critical challenge in event-driven architectures is tracing causality across asynchronous components. To address this, the subsystem implements a dedicated observability pipeline consisting of two specialized workers:

- *The Telemetry Listener:* A service that subscribes to the MQTT bus to ingest system metrics.
- *The Flow Processor:* An asynchronous worker that reconstructs distributed execution traces. It periodically queries "orphaned" execution records from the document store and correlates them using temporal windowing and unique transaction IDs. This allows the system to calculate precise end-to-end network latencies and processing times across the edge-cloud continuum, providing the data necessary to validate the real-time latency requirement (NFR1).

F. Orchestration and Resource Management

The **Orchestrator** subsystem serves as the entry point for requests into the system, enabling the transition from static configurations to dynamic, self-healing environments. It is responsible for the lifecycle management of distributed edge nodes, ensuring that computational resources are allocated efficiently to satisfy the system's real-time latency targets (NFR1).

1) The Control Plane: The subsystem implements a centralized control plane based on the Monitor-Analyze-Plan-Execute (MAPE) loop [34]. It aggregates high-frequency telemetry—including CPU/GPU saturation, memory pressure, and network latency—published by lightweight **Monitor Agents** sidecars deployed on every edge node.

This telemetry flows into the Decision Manager, a heuristic-based policy engine inside the Orchestrator. Unlike static schedulers, this engine evaluates the health of the distributed fleet in real-time. If a node reports resource exhaustion or fails to heartbeat, the Decision Manager triggers dynamic reallocation routines, such as migrating inference tasks to underutilized nodes or, in critical failure scenarios, gracefully degrading non-essential services to preserve core safety functions.

2) Edge Autonomy and Scalability: To support horizontal scalability (NFR3), the architecture employs a Sidecar Pattern [35]. Each processing node runs a local agent that abstracts the hardware specifics from the central orchestrator. These agents utilize MQTT-based service discovery to register themselves upon boot, allowing the system to seamlessly incorporate new compute resources without manual reconfiguration.

3) Resilience Mechanisms: Given the unreliability of residential networks, the subsystem incorporates robust fault-tolerance mechanisms to ensure availability. Communication between the central orchestrator and edge agents is governed by Circuit Breaker patterns and Exponential Backoff retry policies [36]. Messages that fail to be delivered after repeated

TABLE II
HARDWARE CONFIGURATION OF THE HETEROGENEOUS NODES USED IN THE EVALUATION.

Node ID	Processor	Memory	GPU
Peer-01	Intel i9-14900 (32 threads)	32 GiB	RTX 4060
Peer-02	Intel i7-12700 (20 threads)	24 GiB	None (CPU only)
Peer-03	Intel i7-10700 (16 threads)	32 GiB	None (CPU only)

attempts are routed to dead-letter queues for asynchronous analysis, preventing transient network partitions from cascading into system-wide failures. This isolation ensures that even if the orchestration layer becomes temporarily unreachable, the local edge nodes retain their last known good configuration and continue performing local safety checks.

VI. EVALUATION

To evaluate the designed architecture, we tested the performance of image processing through the Computer Vision Service and recommendation generation through the Recommendation Engine (RecSys).

A. Vision inference performance

To validate the system's ability to orchestrate high-bandwidth perception tasks under real-time constraints and varying loads, we conducted an end-to-end performance evaluation of the Computer Vision pipeline. The experiment aimed to isolate the latency overhead introduced by the orchestration layer and to quantify the benefits of horizontal scaling across heterogeneous edge nodes.

1) Experimental Setup and Methodology: The evaluation environment consisted of three heterogeneous edge nodes, detailed in Table II. The diversity in hardware specifications—ranging from high-performance i9 processors with dedicated GPUs to older i7 models—was intentional, designed to test the orchestrator's ability to distribute load across asymmetric compute resources.

The experiment followed a strictly defined request-response cycle. An administrative client published *single-shot* inference commands to the system at controlled rates. Upon receipt, the Orchestrator executed its decision logic (Node Selection), dispatched the task to the selected node's sidecar agent, and triggered the Computer Vision module. The pipeline concluded when the inference metadata (the `vision_response`) was published back to the message bus.

To measure latency without inducing observer effects, we employed an out-of-band MQTT telemetry sniffer. This tool subscribed to all workflow topics, timestamping events at millisecond precision to reconstruct the critical path of each request. We defined *End-to-End Latency* as the total elapsed time between the issuance of the command and the receipt of the inference result.

2) Workload Profile: We executed the experiment across three topological configurations: (1) Single Node, (2) Two Nodes, and (3) Three Nodes. For each topology, we subjected the system to increasing load intensities ranging from 0.25 to 5 Requests Per Second (RPS). Each run consisted of 150 discrete requests, generating a dataset that allowed us to analyze latency distribution and failure rates under stress.

3) Analysis of Orchestration Overhead: A primary concern in microservices architectures is whether the orchestration layer introduces unacceptable latency. Our results indicate that the overhead is negligible. Under low-to-moderate load (0.25–1 RPS), the internal orchestration stages—specifically the transition from *New Request* → *Orchestrator* and *Orchestrator* → *Vision Instance*—averaged approximately 7 ms.

In contrast, the actual inference execution (Vision Request → Response) averaged 1.50 s. Consequently, the orchestration logic accounts for less than 1% of the total transaction time. This confirms that the ARGUS orchestration middleware is lightweight and does not bottleneck the high-value perception tasks.

4) Scalability and Horizontal Elasticity: The system's resilience to load was evaluated by observing latency degradation as request rates increased.

- *Single Node Saturation:* With only one node (Peer-01), the system maintained stability up to 1 RPS. However, at loads exceeding 2 RPS, we observed a significant increase in latency variance (jitter) and a rise in dropped responses, indicating processor saturation.
- *Impact of Horizontal Scaling:* Adding a second and third node dramatically mitigated these saturation effects. In the 3-node configuration, the system sustained loads of 5 RPS with near-zero failure rates and stabilized p95 latency.

These results demonstrate that the orchestrator effectively utilizes available peer nodes to absorb demand. By distributing tasks across the heterogeneous cluster, ARGUS transforms a workload that would saturate a single device into a manageable stream, satisfying the scalability requirement (NFR3).

B. Recommendation performance

To evaluate the scalability of the proactive control loop (FR3), we conducted a load capacity analysis of the Recommendation Engine. Unlike the stateless Computer Vision module, the RecSys component performs *Incremental Learning*, meaning it must update its internal model weights in near real-time while processing incoming events. This experiment aims to quantify the latency trade-offs inherent in this continuous learning approach.

1) Experimental Setup and Hardware: Consistent with the "Edge-Cloud Continuum" architecture proposed in Section 3, the Recommendation Engine was deployed on a centralized private cloud server rather than an edge node. The evaluation environment was hosted on a high-performance server equipped with an AMD EPYC 73F3 processor (16-Core, 3.2 GHz), 62 GB of 3200 MHz DDR4 ECC memory, and NVMe SSD storage (Micron 7450 PRO).

It is important to note that the reference implementation evaluated here operates under a *Sequential Processing Model*. To ensure data consistency during incremental training steps, messages in the current prototype are processed synchronously

in a strict pipeline: *Device Event* $\rightarrow$ *Preprocessing* $\rightarrow$ *Persistence* $\rightarrow$ *Incremental Training* $\rightarrow$ *Inference*.

2) Methodology and Metrics: The experiment simulated a multi-home environment by injecting a controlled stream of synthetic IoT events into the message broker. We fixed the total volume at 1000 messages per trial while varying the message injection rate across five levels: $[0.25, 0.5, 1.0, 2.0, 3.0]$ M/s (messages per second).

We captured four key metrics to assess performance:

- *Logical Processing Latency (T_{proc}):* The time elapsed between data ingestion and the generation of a prediction. This includes the time spent updating the model.
- *Queue Residency Time (T_{queue}):* The duration a message waits in the internal buffer before processing begins.
- *Resource Utilization:* CPU and RAM consumption percentages during the batch execution.

3) Performance Analysis: The results, visualized in Figure 6, reveal a distinct performance boundary dictated by the sequential nature of the implementation.

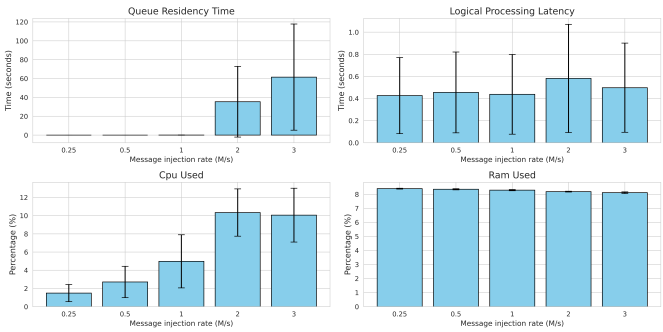

Fig. 6. Performance metrics of the Recommendation Engine under varying load (Injection Rate in M/s). The graphs illustrate the exponential growth in queue time (T_{queue}) beyond the saturation point (2.0 M/s) and the high variance in processing time (T_{proc}) caused by incremental training steps.

a) Saturation Point: At injection rates below 1.0 M/s, the system maintained parity between arrival and service rates. However, at ≥ 2.0 M/s, we observed an exponential growth in T_{queue} (as shown in the top-right quadrant of Figure 6). This indicates that the service rate (μ) of the incremental learning pipeline is approximately 1.5 seconds per message. Once the arrival rate (λ) exceeds this threshold, the system enters a saturated state, creating an unbounded backlog. This confirms that while the logic is sound for single-home deployments, multi-home scalability will require parallelizing the training step in future iterations.

b) Variance and Incremental Learning: A critical finding is the high standard deviation in T_{proc}. This volatility is intrinsic to the *Incremental Learning* requirement (FR3). Periodic model updates are computationally intensive, causing intermittent latency spikes interspersed with fast inference-only cycles. Despite this CPU-bound volatility, RAM usage remained exceptionally stable across all load levels, suggesting that the memory footprint of the online learning algorithms is constant and predictable.

C. Summary

Our evaluation validates **RQ1**, demonstrating that the orchestration middleware introduces negligible overhead ($< 1\%$ of transaction time) and effectively leverages horizontal scaling to stabilize latency under load. Addressing **RQ2**, we identified a critical saturation point of ≈ 1.5 messages per second in the Recommendation Engine, where the coupling of throughput limits and processing latency in the synchronous incremental learning pipeline causes exponential degradation. While this constitutes a bottleneck for individual instances, the ARGUS Orchestrator overcomes this limitation by horizontally sharding recommendation services (*e.g.*, per House ID) across distributed containers, ensuring ecosystem-wide real-time responsiveness despite the sequential constraints of the underlying learning algorithm.

VII. Conclusion and Future Work

This paper presented ARGUS, a context-aware software architecture designed to orchestrate heterogeneous AI subsystems within the Edge-Cloud continuum. The research validated that a distributed, event-driven approach successfully reconciles the conflicting requirements of processing high-bandwidth sensor streams and maintaining low-latency device control. By enforcing privacy-by-design through local edge processing of sensitive video data, the architecture ensures resident privacy without compromising perception capabilities. The experimental evaluation of the orchestration subsystem demonstrated that the system introduces negligible overhead and scales effectively. The addition of edge nodes significantly reduced latency variance and eliminated failures under load, confirming the resilience and scalability of the proposed solution. Furthermore, the integration of Large Language Models marks a significant evolution from static automation rules to proactive, agentic reasoning capable of interpreting natural language and dynamically adapting to user needs.

Future research will address the limitations identified in the current implementation. First, the sequential processing bottleneck observed in the Recommendation Engine at higher loads will be mitigated by refactoring the module into a fully asynchronous, parallelized architecture, thereby enhancing throughput. Second, while the architecture supports distributed deployment, the integration of Federated Learning will be prioritized to enable privacy-preserving model updates across different environments. Third, the orchestration logic will be extended to incorporate energy consumption metrics, optimizing workload distribution for both power efficiency and latency. Finally, the capabilities of the AI agents will be expanded to support long-horizon planning and deeper multi-modal integration, such as Visual Question Answering, to improve robustness in ambiguous real-world scenarios.

References

[1] C. Ramos, J. C. Augusto, and D. Shapiro, "Ambient intelligence—the next step for artificial intelligence," *IEEE Intelligent Systems*, vol. 23, no. 2, pp. 15–18, 2008.

[2] K. J. Kaaz, A. Hoffer, M. Saeidi, A. Sarma, and R. B. Bobba, "Understanding user perceptions of privacy, and configuration challenges in home automation," in *2017 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, 2017, pp. 297–301.

[3] W. Fang and C. Zhu, "Advances in architecture, protocols, and challenges in internet of things: From technologies to applications," *Applied Sciences*, vol. 14, no. 8, 2024. [Online]. Available: https://www.mdpi.com/2076-3417/14/8/3266

[4] W. Li, T. Yigitcanlar, I. Erol, and A. Liu, "Motivations, barriers and risks of smart home adoption: From systematic literature review to conceptual framework," *Energy Research & Social Science*, vol. 80, p. 102211, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2214629621003042

[5] C. Carra and K. Tabia, "Smart Home for Seniors: Opportunities and Challenges for AI," in *ICAART 2020, 12th International Conference on Agents and Artificial Intelligence*, Valetta, Malta, Feb. 2020. [Online]. Available: https://hal.science/hal-02530118

[6] D. Campos, L. Martins, J. Mota, D. Tavares, J. Pereira, M. Oliveira, D. Boaventura, D. Correa, E. Ferreira, G. Pinto, N. Seixas, A. Maia, M. Romário, E. Passos, F. Durao, G. B. Figueiredo, M. Peixoto, T. Januario, C. Prazeres, I. Machado, and E. Almeida, "Designing, implementing, and testing ai-oriented smart home applications: Challenges and best practices," in *Software Architecture. ECSA 2024 Tracks and Workshops*, A. Ampatzoglou, J. Pérez, B. Buhnova, V. Lenarduzzi, C. C. Venters, U. Zdun, K. Drira, L. Rebelo, D. Di Pompeo, M. Tucci, E. Y. Nakagawa, and E. Navarro, Eds. Cham: Springer Nature Switzerland, 2024, pp. 83–99.

[7] B. K. Sovacool and D. D. Furszyfer Del Rio, "Smart home technologies in europe: A critical review of concepts, benefits, risks and policies," *Renewable and Sustainable Energy Reviews*, vol. 120, p. 109663, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1364032119308688

[8] P. Gkonis, A. Giannopoulos, P. Trakadas, X. Masip-Bruin, and F. D'Andria, "A survey on iot-edge-cloud continuum systems: Status, challenges, use cases, and open issues," *Future Internet*, vol. 15, no. 12, p. 383, 2023.

[9] F. Alkhabbas, R. Spalazzese, M. Cerioli, M. Leotta, and G. Reggio, "On the deployment of iot systems: An industrial survey," in *2020 IEEE International Conference on Software Architecture Companion (ICSA-C)*, 2020, pp. 17–24.

[10] M. Fowler and J. Lewis, "Microservices: a definition of this new architectural term," *martinfowler.com*, March 2014, accessed: 2025-11-14. [Online]. Available: https://martinfowler.com/articles/microservices.html

[11] C. Richardson, *Microservices Patterns: With examples in Java*, 1st ed. Manning Publications, 2018.

[12] L. Martins, D. Campos, J. Mota, D. Tavares, J. Pereira, M. Oliveira, D. Boaventura, D. Correa, E. Ferreira, G. Pinto, N. Seixas, A. Maia, M. Romário Pinheiro dos Santos, E. Batista, F. Durão, G. Figueiredo, M. Peixoto, C. Prazeres, I. Machado, and E. Almeida, "A case study of smart home development," *IEEE Software*, vol. PP, pp. 1–7, 01 2024.

[13] Y. M. Leong, E. H. Lim, and L. K. Lim, "A review of potential ai-based automation for iot-enabled smart homes," in *2023 IEEE 13th International Conference on System Engineering and Technology (ICSET)*, 2023, pp. 1–6.

[14] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, W. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, "The rise and potential of large language model based agents: A survey," *CoRR*, vol. abs/2309.07864, 2023. [Online]. Available: https://arxiv.org/abs/2309.07864

[15] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," *CoRR*, vol. abs/2201.11903, 2022. [Online]. Available: https://arxiv.org/abs/2201.11903

[16] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, "Language agent tree search unifies reasoning, acting, and planning in language models," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://arxiv.org/abs/2310.04406

[17] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[18] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *CoRR*, vol. abs/2302.04761, 2023. [Online]. Available: https://arxiv.org/abs/2302.04761

[19] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, ser. UIST '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3586183.3606763

[20] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, and J.-R. Wen, "A survey on the memory mechanism of large language model based agents," *CoRR*, vol. abs/2404.13501, 2024. [Online]. Available: https://arxiv.org/abs/2404.13501

[21] H. H. Hassan, G. Bouloukakis, L. Scalzotto, N. Khaled, D. Conan, A. Kattepur, and D. Belaïd, "A message broker architecture for adaptive data exchange in the iot," in *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*, 2024, pp. 151–158.

[22] D. C. da Silva, D. Robson Dantas Boaventura, M. dos Santos Oliveira, J. Pereira Santos Junior, E. Ferreira da Silva, E. S. de Almeida, C. V. S. Prazeres, I. do Carmo Machado, M. Leone Maciel Peixoto, G. Bittencourt Figueiredo, and F. A. Durão, "Evaluating multi-label machine learning models for smart home environments," *Software: Practice and Experience*, vol. 55, no. 9, pp. 1427–1444, 2025. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.3428

[23] M. Richards and N. Ford, *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, 2020. [Online]. Available: https://books.google.com.br/books?id=xa7MDwAAQBAJ

[24] R. A. Light, "Mosquitto: server and client implementation of the MQTT protocol," *Journal of Open Source Software*, vol. 2, no. 13, p. 265, 2017. [Online]. Available: https://doi.org/10.21105/joss.00265

[25] MongoDB, Inc., *MongoDB*, MongoDB, Inc., 2025, general Purpose, Document-Based, Distributed Database. [Online]. Available: https://www.mongodb.com/

[26] Redis Ltd., "Redis," https://redis.io/, 2025, in-memory data structure store.

[27] MinIO, Inc., "Minio high performance object storage," 2025, multi-Cloud Object Storage. [Online]. Available: https://min.io/

[28] MariaDB Foundation, *MariaDB Server*, 2025, open source relational database. [Online]. Available: https://mariadb.org/

[29] I. Sysoev and F5, Inc., *NGINX: High Performance Load Balancer, Web Server, & Reverse Proxy*, 2025, open source web server. [Online]. Available: https://nginx.org/

[30] A. Ronacher and Pallets Projects, "Flask," 2025, a lightweight WSGI web application framework. [Online]. Available: https://flask.palletsprojects.com/

[31] H. Soudani, E. Kanoulas, and F. Hasibi, "Fine tuning vs. retrieval augmented generation for less popular knowledge," in *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*, ser. SIGIR-AP 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 12–22. [Online]. Available: https://doi.org/10.1145/3673791.3698415

[32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 6000–6010.

[33] S. Ramírez, "Fastapi," 2025, a modern, fast (high-performance), web framework for building APIs with Python. [Online]. Available: https://fastapi.tiangolo.com/

[34] H. Jilderda and C. Raibulet, "Mape-k based guidelines for designing reactive and proactive self-adaptive systems," in *Software Architecture. ECSA 2023 Tracks, Workshops, and Doctoral Symposium*, B. Tekinerdoğan, R. Spalazzese, H. Sözer, S. Bonfanti, and D. Weyns, Eds. Cham: Springer Nature Switzerland, 2024, pp. 53–68.

[35] B. Burns and D. Oppenheimer, "Design patterns for container-based distributed systems," in *8th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 16)*. Denver, CO: USENIX Association, Jun. 2016. [Online]. Available: https://www.usenix.org/conference/hotcloud16/workshop-program/presentation/burns

[36] M. Nygard, *Release It!: Design and Deploy Production-ready Software*, ser. Pragmatic programmers. Pragmatic Bookshelf, 2018. [Online]. Available: https://books.google.com.br/books?id=UW7jAQAACAAJ